

Java Multithreading and File Handling(part 3): Viva Answer Guide (Q1–30)

Concise, viva-ready answers on multithreading (Q1–15) and file handling (Q16–30).

A. Multithreading in Java (Q1–15)

1. What is a thread in Java? A thread is the smallest unit of execution within a program, a lightweight path of execution. Every Java program starts with one thread, the `main` thread.

2. What is multithreading? Running two or more threads concurrently within the same program so that tasks can progress at the same time (or appear to, by sharing CPU time).

3. Why is multithreading used in Java applications? To make better use of the CPU, keep applications responsive (e.g. a UI that does not freeze during a download), run background tasks, and improve performance by splitting work into parallel tasks.

4. What is the difference between a process and a thread?

Process	Thread
Independent program in execution	Lightweight unit within a process
Has its own separate memory	Shares the memory of its process
Heavyweight; context switching is costly	Lightweight; context switching is cheap
Communication between processes is difficult	Threads communicate easily via shared data

5. How can you create a thread in Java? Two main ways:

1. Extend the `Thread` class and override `run()`.
2. Implement the `Runnable` interface and pass it to a `Thread` object.

```
// Way 1: extending Thread
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}

// Way 2: implementing Runnable
class MyTask implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
}

public class Demo {
    public static void main(String[] args) {
        new MyThread().start();
    }
}
```

```

        new Thread(new MyTask()).start();
    }
}

```

6. What is the difference between extending Thread and implementing Runnable?

Extending Thread	Implementing Runnable
Class cannot extend any other class	Class can still extend another class
Tightly couples task and thread	Separates the task from the thread
Each thread has its own object	Same Runnable object can be shared by many threads
Less flexible	Preferred approach

7. What is the purpose of the start() method? start() creates a new thread of execution and makes the JVM call the thread's run() method in that new thread. A thread can be started only once; calling start() again throws `IllegalThreadStateException`.

8. What is the difference between start() and run()? start() creates a new thread and runs run() in it (true multithreading). Calling run() directly is just an ordinary method call executed in the current thread, so no new thread is created.

```

t.start(); // new thread runs run()
t.run();  // runs in the main thread, no new thread

```

9. What is the life cycle of a thread? A thread passes through these states:

1. **New:** thread object created, start() not yet called.
2. **Runnable:** start() called; ready to run or running (waiting for the CPU scheduler).
3. **Blocked:** waiting to acquire a monitor lock.
4. **Waiting / Timed Waiting:** waiting for another thread (wait(), join()) or for a fixed time (sleep(ms)).
5. **Terminated (Dead):** run() has finished or the thread was stopped by an uncaught exception.

10. What is the purpose of sleep()? Thread.sleep(milliseconds) pauses the currently executing thread for the given time without releasing any locks it holds. It throws `InterruptedException`, which must be handled.

```

try {
    Thread.sleep(1000); // pause 1 second
} catch (InterruptedException e) {
    e.printStackTrace();
}

```

11. What is the difference between sleep() and wait()?

sleep()	wait()
Method of Thread class (static)	Method of Object class
Does not release the lock	Releases the lock

sleep()	wait()
Wakes up after the time expires	Wakes up on notify()/notifyAll() (or timeout)
Can be called anywhere	Must be called inside a synchronized block or method

12. What is thread synchronization? Controlling access to shared resources so that only one thread at a time can use them, which keeps data consistent and prevents interference between threads.

13. Why is the synchronized keyword used? It makes a method or block acquire the object's lock (monitor) before executing, so only one thread can run that code on the object at a time. It prevents race conditions and data inconsistency.

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public int getCount() { return count; }
}
```

14. What is a race condition? A situation where the result depends on the unpredictable timing or order in which multiple threads access and modify shared data. For example, two threads doing count++ on the same variable at once can lose an update, because count++ is read, add, write rather than a single step. It is fixed with synchronization.

15. What is the difference between join() and sleep()?

join()	sleep()
Makes the current thread wait until another thread finishes	Pauses the current thread for a fixed time
Instance method, called as t.join()	Static method, Thread.sleep(ms)
Depends on another thread's completion	Depends only on time
Throws InterruptedException	Throws InterruptedException

```
Thread t = new Thread(() -> System.out.println("Working..."));
t.start();
t.join(); // main waits here until t completes
System.out.println("Done");
```

B. File Handling in Java (Q16–30)

16. What is file handling in Java? The process of creating, reading, writing, updating and deleting files (and working with their properties) from a Java program, so data can be stored permanently, beyond the program's run.

17. Which package is commonly used for file handling in Java? The `java.io` package (classes such as `File`, `FileReader`, `FileWriter`, `BufferedReader`, `FileInputStream`, `FileOutputStream`). The newer `java.nio.file` package (`Files`, `Path`) is also used.

18. What is the purpose of the `File` class? The `File` class represents a file or directory path. It is used to create, delete, rename, check existence and obtain information (name, path, size, permissions). It does not read or write file contents.

19. How do you create a new file using Java? Use `createNewFile()` of the `File` class. It returns `true` if the file was created and `false` if it already exists, and it throws `IOException`.

```
import java.io.File;
import java.io.IOException;

public class CreateFile {
    public static void main(String[] args) {
        try {
            File f = new File("demo.txt");
            if (f.createNewFile()) {
                System.out.println("File created: " + f.getName());
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

20. How do you check whether a file exists? Use the `exists()` method, which returns `true` or `false`. Related methods: `isFile()`, `isDirectory()`, `canRead()`, `canWrite()`.

```
File f = new File("demo.txt");
System.out.println(f.exists());
```

21. How do you obtain the name and path of a file?

```
File f = new File("demo.txt");
System.out.println(f.getName());           // file name
System.out.println(f.getPath());           // path as given
System.out.println(f.getAbsolutePath());   // full path
System.out.println(f.length());            // size in bytes
```

22. What is the difference between `FileReader` and `FileWriter`? `FileReader` reads characters from a text file (input). `FileWriter` writes characters to a text file (output). `FileWriter` creates the file if it does not exist and overwrites it by default; use `new FileWriter(name, true)` to append.

23. What is the difference between character streams and byte streams?

Byte streams	Character streams
Handle data as 8-bit bytes	Handle data as 16-bit Unicode characters
<code>InputStream</code> / <code>OutputStream</code> classes	<code>Reader</code> / <code>Writer</code> classes
Used for binary data (images, audio, PDFs)	Used for text data

Byte streams	Character streams
Examples: <code>FileInputStream</code> , <code>FileOutputStream</code>	Examples: <code>FileReader</code> , <code>FileWriter</code>

24. What is the purpose of `FileInputStream`? `FileInputStream` reads raw bytes from a file, so it is suitable for binary files such as images. Its `read()` method returns the next byte as an int, or -1 at the end of the file.

```
try (FileInputStream fis = new FileInputStream("data.bin")) {
    int b;
    while ((b = fis.read()) != -1) {
        System.out.print((char) b);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

25. What is the purpose of `FileOutputStream`? `FileOutputStream` writes raw bytes to a file. It creates the file if it does not exist and overwrites it unless opened in append mode.

```
try (FileOutputStream fos = new FileOutputStream("data.bin")) {
    fos.write("Hello".getBytes());
} catch (IOException e) {
    e.printStackTrace();
}
```

26. What is the difference between `BufferedReader` and `FileReader`? `FileReader` reads characters directly from the file, one or a few at a time, which is slow. `BufferedReader` wraps a reader and uses an internal buffer, so it reads in large chunks (faster) and adds the `readLine()` method to read a full line.

27. How do you read a file line by line in Java? Wrap a `FileReader` in a `BufferedReader` and call `readLine()` until it returns null.

```
import java.io.*;

public class ReadLines {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("demo.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

28. How do you write data into a text file? Use `FileWriter` (optionally wrapped in `BufferedWriter`) and call `write()`.

```
import java.io.*;

public class WriteFile {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("demo.txt")) {
```

```

        fw.write("Hello, Java File Handling!");
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

29. Why should files be closed after reading or writing? Closing a file releases system resources (file handles and memory), prevents resource leaks and file locks, and, for writers, flushes buffered data so it is actually saved to disk. Use `close()` in a `finally` block, or better, `try-with-resources`, which closes the resource automatically.

30. What exceptions can occur during file handling, and how can they be handled?

- `IOException`: general input/output failure (checked).
- `FileNotFoundException`: file does not exist or cannot be opened (a subclass of `IOException`).
- `SecurityException`: no permission to access the file (unchecked).

They are handled with `try-catch` (or declared with `throws`), and resources are closed with `finally` or `try-with-resources`.

```

try (BufferedReader br = new BufferedReader(new FileReader("missing.txt"))) {
    System.out.println(br.readLine());
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("I/O error: " + e.getMessage());
}

```